SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Descriptive Written Exam

Name :- Shaik Ashraf

Register Number :- 192321085

COURSE NAME : Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3

3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3

8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3

15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I **BHAVNANI SANKAR (192411238)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

BSPNAIDU.//

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

1. Testing as an Engineering Activity

Testing should be treated as a systematic engineering activity rather than random or ad-hoc testing.

- First, analyze the banking requirements and identify critical functions.
- Prepare a proper test plan containing scope, resources, schedule, risks, and responsibilities.
- Design test cases systematically using equivalence partitioning, boundary value analysis, decision tables, and state-transition testing.
- Define expected results before executing the test cases.
- Use test-management and defect-tracking tools to maintain test information.
- Follow common standards for test cases, defect severity, priority, and reporting.
- Perform functional, integration, security, performance, and regression testing.
- Replace ad-hoc testing with risk-based and structured testing.
- For example, fund transfer should be tested for valid transactions, insufficient balance, invalid account numbers, transaction limits, duplicate transactions, and network failures.
- After defects are fixed, perform retesting and regression testing.
- This approach improves test coverage, defect detection, reliability, consistency, and software quality.

2. Structured Testing Process for ATM

A structured testing process should be followed throughout the Software Development Life Cycle (SDLC).

- During requirement analysis, identify all ATM withdrawal requirements.
- During test planning, define the testing scope, resources, risks, and schedule.

- During test design, prepare positive, negative, and boundary test cases.
- During test execution, compare actual results with expected results.
- Record every failure in a defect repository.
- Classify defects according to severity and priority.
- After fixing a defect, perform retesting to verify the correction.
- Perform regression testing to ensure that the correction has not affected other ATM functions.
- ATM test cases include valid withdrawal, incorrect PIN, insufficient balance, invalid amount, daily withdrawal limit, and insufficient cash.
- A structured process provides consistency across all testing activities.
- It reduces defects and improves ATM reliability and customer satisfaction.

3. Error, Defect and Failure

Error, defect, and failure are related but different concepts in software testing.

- An error is a human mistake made during requirements, design, coding, or testing.
- A defect is a fault introduced into the software because of an error.
- A failure occurs when the software executes a defect and produces an incorrect result.
- Example: A programmer incorrectly writes the password validation condition.
- The programmer's mistake is the error.
- The incorrect condition in the program is the defect.
- When a valid user is unable to log in, it becomes a failure.
- The same concept applies to transaction operations when the actual transaction result differs from the expected result.
- Defect reports should clearly mention steps to reproduce, expected result, actual result, severity, and environment.

- Proper understanding of these terms improves communication between testers and developers.
- Clear terminology helps in faster defect identification, tracking, and resolution.

4. Defect Clustering and Test Prioritization

Defect clustering means that a small number of modules may contain a large number of defects.

- Historical defect information can be used to identify defect-prone modules.
- Testing should give higher priority to modules having frequent or serious defects.
- Business-critical functionality should also receive higher testing priority.
- Test cases should be prioritized according to risk, complexity, business importance, and defect history.
- Recently modified modules should receive additional regression testing.
- For example, in an e-commerce application, payment and checkout modules are more critical than a simple profile page.
- High-priority test cases should be executed first.
- Low-risk functions can be tested after critical functions.
- Defect clustering helps testers use limited time and resources effectively.
- Risk-based prioritization improves defect detection and reduces testing delays.
- Therefore, applying testing principles makes the testing process more efficient and focused.

5. Defect Classification and Defect Repository

Defects can originate at different stages of software development.

- Common sources include requirements, design, coding, integration, and configuration.
- Defects can be classified as functional, performance, security, usability, interface, compatibility, and data-related defects.

- A defect repository stores complete information about identified defects.
- Important information includes defect ID, description, module, severity, priority, status, root cause, and resolution.
- Historical defect information helps identify repeated problems.
- If the same defect occurs in multiple releases, testers can identify a pattern from repository data.
- New regression test cases can be created based on previously detected defects.
- Root-cause analysis should be performed to determine why the defect occurred.
- Code reviews and preventive testing can reduce repeated defects.
- Regular analysis of the repository improves future test planning.
- Effective use of defect history improves software quality and prevents recurring problems.

6. Mobile Banking Fund Transfer

A mobile banking application that frequently crashes during fund transfer requires systematic testing.

- First, reproduce the crash and collect error logs and other relevant information.
- Check the application version, device, operating system, network, and transaction data.
- Test normal fund-transfer operations.
- Test negative conditions such as insufficient balance and invalid account numbers.
- Test large transaction amounts and transaction limits.
- Test network interruption during a transaction.
- Test session timeout and multiple transactions.
- Perform integration testing between the application, database, and banking services.
- Record every failure in a defect repository with proper severity and priority.

- After fixing the defect, perform retesting and regression testing.
- Systematic testing helps identify the root cause and prevents similar crashes in future releases.
- It ultimately improves application stability, reliability, and customer confidence.

7. Testing and Debugging

Testing and debugging are related activities but have different purposes.

- Testing is performed to identify defects and verify whether software meets its requirements.
- Debugging is performed to locate and correct the cause of a detected defect.
- Testing generally identifies that a problem exists.
- Debugging identifies where the problem exists and how to fix it.
- For example, testing may identify that the balance after a fund transfer is incorrect.
- The developer then debugs the program to locate the incorrect calculation.
- After the correction, testers perform retesting.
- Regression testing is then performed to ensure that other functions are not affected.
- Confusing testing with debugging can cause poor defect tracking and inefficient resolution.
- The proper process is Testing → Defect Reporting → Debugging → Retesting → Regression Testing.
- This separation makes software quality activities more effective.

8. Cross-Browser Testing

Cross-browser testing ensures that a web application behaves consistently across different browsers.

- Test the application using browsers such as Chrome, Firefox, Edge, and Safari, where applicable.
- Check different browser versions and operating systems when required.
- Test login, forms, navigation, transactions, buttons, and other important functions.
- Check whether the user interface is displayed correctly.
- Browser-specific JavaScript and CSS differences can cause defects.
- Responsive behavior should also be tested on different screen sizes.
- Functional testing should be combined with compatibility testing.
- Automation tools such as Selenium can improve the efficiency of repeated browser tests.
- A browser compatibility matrix can help testers organize test coverage.
- Defects should be recorded with browser and operating-system information.
- Systematic cross-browser testing improves compatibility, reliability, and user experience.

9. Payroll Logical Errors

A logical error occurs when software executes successfully but produces an incorrect result.

- Payroll systems can contain defects in salary, overtime, tax, deductions, and bonus calculations.
- Such defects may be classified as calculation, business-rule, data, or requirement defects.
- Testers should understand the defect class to select suitable testing techniques.
- Normal salary values should be tested.
- Boundary values such as minimum and maximum salary should also be tested.
- Test overtime, deductions, tax, bonuses, and different employee categories.
- Unit testing can detect calculation errors early.

- Code reviews can help identify incorrect business logic.
- Boundary-value analysis and equivalence partitioning improve test coverage.
- Regression testing should be performed after fixing logical errors.
- Proper defect classification helps prevent similar calculation errors in future releases.

10. Structured Test Case Design

Random testing is not sufficient for achieving good test coverage.

- Testing should start from software requirements and identified risks.
- A test case should contain test ID, objective, preconditions, input, steps, expected result, actual result, and status.
- Equivalence partitioning divides inputs into valid and invalid groups.
- Boundary value analysis tests values around input limits.
- Decision-table testing is useful for multiple business conditions.
- State-transition testing is useful when system behavior changes between states.
- Risk-based testing gives priority to critical functions.
- For example, a banking transaction limit should be tested below, at, and above the allowed limit.
- Structured test cases make testing repeatable and easier to review.
- They improve requirement coverage and defect detection.
- Therefore, systematic test-case design is more effective than random testing.

11. Healthcare Data Inconsistency

Data inconsistency can occur when different modules do not correctly maintain or update the same information.

- Defects may originate from incorrect requirements, coding mistakes, database problems, or integration issues.
- Testing should verify that data remains consistent across all modules.
- Patient information can be checked between registration, medical records, billing, and reporting.
- Database testing should verify correct storage and retrieval of information.
- Integration testing should verify communication between different modules.
- Data validation should be performed for valid and invalid inputs.
- Transaction testing should verify that updates are correctly completed.
- Defects should be recorded with module, expected result, actual result, and severity.
- Regression testing should be performed after fixing data-related defects.
- Proper defect tracking makes recurring data problems easier to identify.
- Systematic testing ensures data integrity and improves healthcare application reliability.

12. Defect Repository and Repeated Defects

A defect repository is useful for storing and analyzing defects found during different software releases.

- It should contain defect ID, module, description, type, severity, priority, root cause, and resolution.
- Historical repository data can show which modules produce the most defects.
- Defects can also be analyzed according to release and defect type.
- Repeated defects indicate that the original root cause may not have been properly addressed.
- Testers should convert important historical defects into regression test cases.
- Frequently failing modules should receive additional testing.
- Root-cause analysis should be performed for recurring problems.

- Code reviews can help prevent similar defects during development.
- Automated regression testing can verify that old defects do not return.
- Repository analysis helps improve future test planning.
- Proper use of defect history reduces recurring defects and improves software quality.

13. Peak-Usage Failure in Learning System

A web-based learning system that fails during peak usage indicates insufficient performance testing.

- Testing should identify expected normal and maximum numbers of users.
- Load testing checks application behavior under expected workload.
- Stress testing checks the system beyond its normal capacity.
- Performance testing measures response time, throughput, and resource usage.
- Test multiple users accessing the system simultaneously.
- Important functions such as login, course access, examination, and result submission should be tested under load.
- Database and server performance should also be monitored.
- For example, if thousands of students access an examination system simultaneously, this scenario should be simulated before deployment.
- Test coverage should include peak and abnormal usage conditions.
- Performance defects should be recorded and analyzed.
- Better test planning and performance coverage improve reliability and prevent failures after deployment.

14. Early Tester Involvement in SDLC

Testers should be involved from the early stages of software development.

- During requirements analysis, testers can identify incomplete or unclear requirements.
- During design, they can identify risks and testability issues.
- During development, testers can support unit and integration testing.
- Testers can prepare test cases before the complete software is developed.
- Early defect detection reduces the cost and time required for fixing defects.
- Regular communication between developers, testers, business analysts, and stakeholders is important.
- For example, in a banking application, testers can identify security and transaction requirements before coding.
- Early reviews can prevent incorrect requirements from reaching later stages.
- Tester involvement also improves test planning and coverage.
- It reduces rework and improves product quality.
- Therefore, testing should be considered throughout the SDLC rather than only at the end.

15. Incomplete Test Coverage

Incomplete test coverage means that some requirements, conditions, inputs, or paths are not tested.

- Test coverage should be planned according to requirements and risks.
- Functional and non-functional requirements should be covered.
- Positive and negative scenarios should both be included.
- Equivalence partitioning can cover different classes of input.
- Boundary value analysis can identify defects at limits.
- Edge cases and invalid inputs should also be included.
- For example, an ATM withdrawal should test valid amount, zero, negative amount, maximum limit, and insufficient balance.

- Regression testing ensures previously working functionality continues to work.
- Test coverage should be reviewed before test closure.
- Missing coverage can result in failures after deployment.
- Proper coverage increases confidence, reliability, and defect detection.

16. Defect Reporting

Good defect reporting helps developers understand and correct problems quickly.

- Each defect should have a unique defect ID.
- The report should include a clear title and module name.
- Provide the environment and application version.
- Mention the exact steps required to reproduce the problem.
- Clearly state the expected result and actual result.
- Specify defect severity and priority.
- Attach screenshots, logs, or other evidence when necessary.
- Example: “Transaction fails when amount exceeds the permitted limit” is clearer than “Transaction problem.”
- A centralized defect-tracking system should be used.
- Standard reporting formats should be followed by all testers.
- Clear reports reduce misunderstanding and improve communication.
- Proper reporting also makes defect tracking and resolution easier.

17. Process-Based Testing

Process-based testing ensures that testing activities are performed consistently across different modules.

- A standard testing process should be defined for the entire project.

- The process can include Requirement Analysis → Test Planning → Test Design → Execution → Defect Reporting → Retesting → Regression Testing → Closure.
- All testers should follow a common test-case format.
- Common definitions should be used for defect severity and priority.
- Test plans and test cases should be reviewed before execution.
- Standard procedures reduce differences between testers.
- For example, all modules in a banking system can follow the same procedure for recording and resolving defects.
- Checklists can be used to ensure important activities are not missed.
- Test metrics can be used to monitor progress and quality.
- Standardized processes improve consistency, traceability, repeatability, and quality.
- Process-based testing also makes testing easier to manage across multiple modules.

18. Boundary Testing and Edge Cases

Edge cases are special conditions that occur at the limits of input or system behavior.

- Such cases are often missed when testers focus only on normal inputs.
- Boundary Value Analysis is an important technique for detecting boundary-related defects.
- Test values just below, at, and just above the boundary.
- For example, if an application accepts age from 18 to 60, test 17, 18, 19, 59, 60, and 61.
- Similar testing can be applied to transaction limits, file sizes, quantities, and dates.
- Minimum and maximum values should always be considered.
- Zero, empty, invalid, and unusually large inputs can also be tested where appropriate.
- Edge cases should be included during test-case design rather than after deployment.

- Boundary testing improves coverage and identifies defects that normal testing may miss.
- It is especially useful for systems having strict input limits.
- Including edge cases helps prevent failures after deployment.

19. Defect Documentation and Repository

A properly maintained defect repository makes defect management easier.

- Every defect should have a unique ID and clear description.
- The repository should contain module, environment, steps, expected result, actual result, severity, priority, and status.
- Root cause and resolution should also be documented.
- Defects can be tracked from New → Assigned → Fixed → Retested → Closed.
- Historical records help identify frequently failing modules.
- Repeated defects can be analyzed to identify common causes.
- Previous defects can be converted into regression test cases.
- Repository data can support future test planning and prioritization.
- Proper documentation prevents loss of defect information between releases.
- It also improves communication between testers and developers.
- Therefore, a structured repository supports analysis, prevention, tracking, and continuous quality improvement.

20. Banking Transaction Processing

Banking transaction processing requires systematic and well-planned testing because transaction failures can affect customers and financial data.

- Begin with requirement analysis and identify transaction-related risks.

- Prepare positive and negative test cases.
- Test successful transactions, insufficient balance, invalid account details, and transaction limits.
- Test duplicate transactions and network interruptions.
- Test timeout and session-related conditions.
- Use equivalence partitioning, boundary value analysis, decision tables, and state-transition testing.
- Perform integration testing between the application, database, and transaction services.
- Security testing should verify authentication and authorization requirements.
- Defects should be recorded with clear severity, priority, and reproduction steps.
- After fixing defects, perform retesting and regression testing.
- Automated regression tests can improve repeated testing.
- Structured testing improves coverage, defect detection, transaction reliability, and overall software quality.